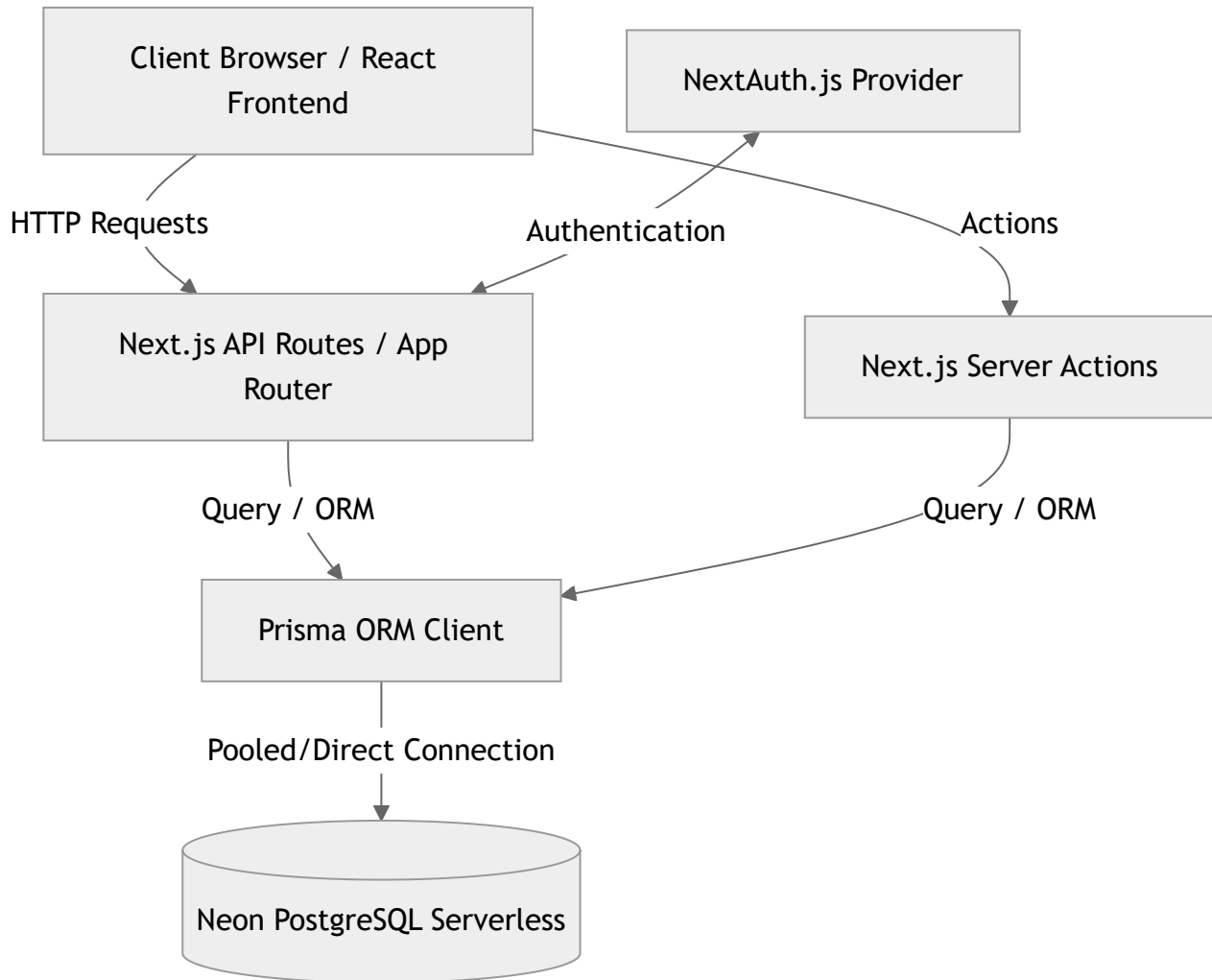


JetLabs Backend Architecture and Workflow

This document provides a comprehensive technical overview of the backend architecture, data model, execution lifecycles, and performance optimizations for the JetLabs academic project execution platform.

1. Technical Architecture Overview

JetLabs is built on a modern, serverless-ready Node.js stack using Next.js, Prisma ORM, and PostgreSQL.

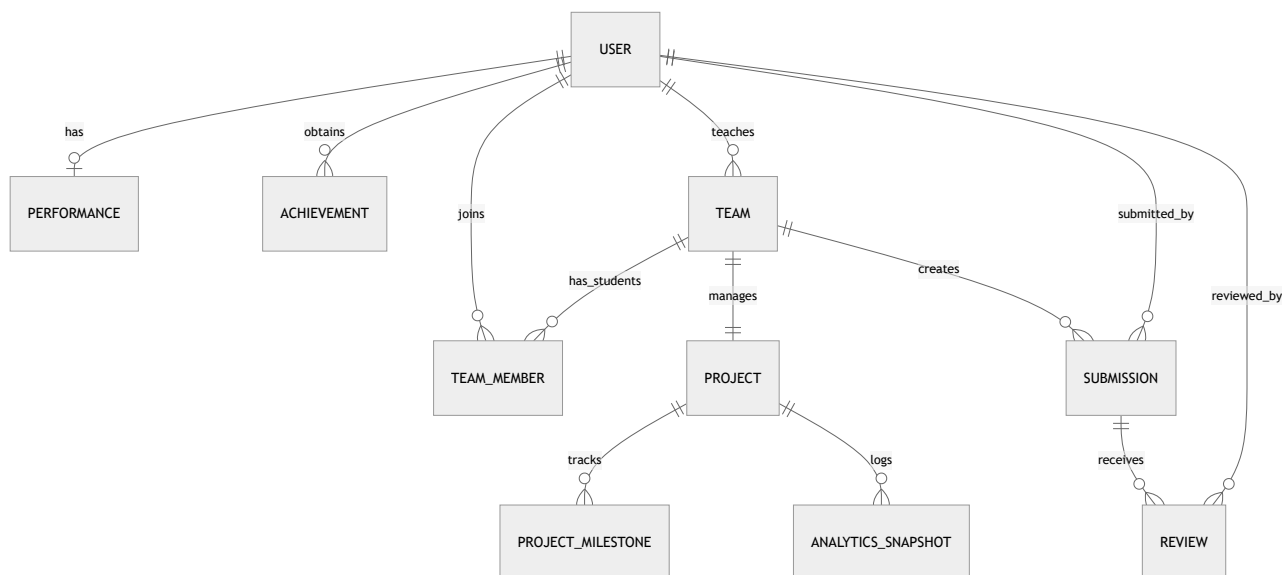


Core Technologies

- **Framework:** Next.js (App Router, Node.js runtime).
- **Database ORM:** Prisma Client (v6.x) with Relation Joins enabled.
- **Database:** Neon serverless PostgreSQL, utilizing PgBouncer connection pooling.
- **Security & Auth:** NextAuth.js (JWT session strategy, bcryptjs password hashing, and custom middleware authorization).
- **Validation:** Zod (unified schema validation across server and client).

2. Database Schema (Prisma Data Model)

The database schema is organized around three primary hubs: **Users**, **Teams & Projects**, and **Submissions & Reviews**.



Core Entity Definitions

A. User & Authentication

- User** : The central actor. Includes credentials (`passwordHash`), roles (`STUDENT` , `FACULTY` , `ADMIN`), and department metadata.
- Performance** : Academic records containing `score` , `attendanceScore` , `submissionScore` , and `reviewScore` . Linked to **User** via a `1:1` relationship.
- Achievement** : Gamification achievements containing `points` and `badge` types linked to a user.

B. Team & Execution

- Team** : Group of student members assigned to a specific batch and guided by a faculty advisor.
- TeamMember** : A join table mapping **User** (Students) to **Team** records, carrying a `contributionScore` and `lastActiveAt` timestamp.
- Project** : The workspace of a team, carrying progress percentages, status trackers (`DISCOVERY` to `COMPLETED`), and overall project risk scores.

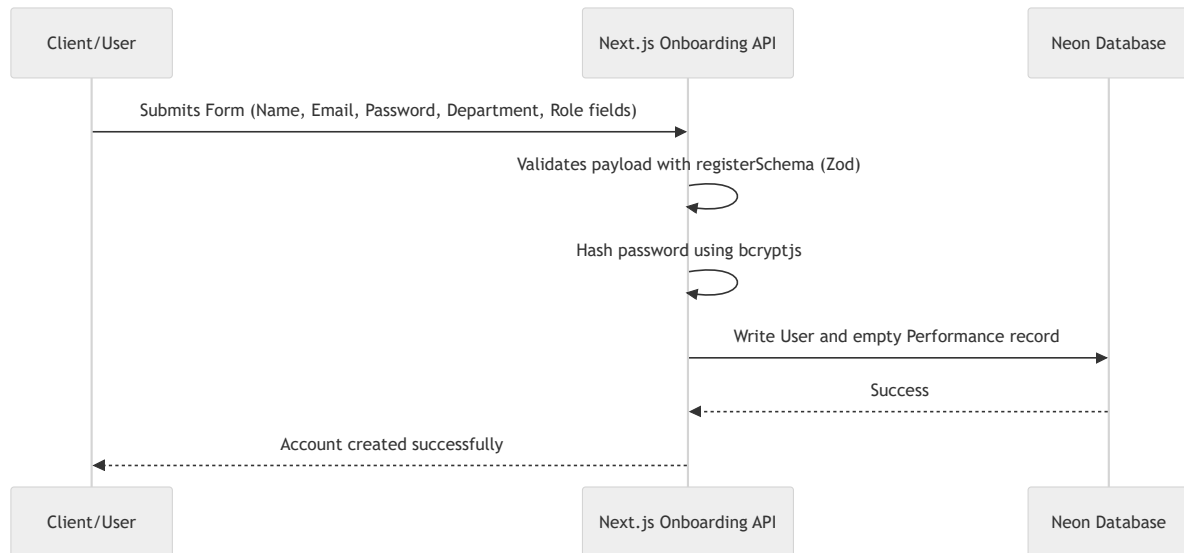
- **ProjectMilestone** : Individual milestone tasks within a project stage (e.g., **Research** , **Execution** , **Evaluation**) containing statuses (**PENDING** , **IN_PROGRESS** , **COMPLETED** , **BLOCKED**).

C. Submissions & Evaluation

- **Submission** : File uploads or text summaries submitted by a team for review. Classified by **SubmissionType** (**WEEKLY** , **FINAL** , **IEEE** , **LITERATURE**) and **SubmissionStatus** .
 - **Review** : Faculty-generated decision records linked to a submission. Contains a score, comment text, and decision status (**APPROVED** , **REVISION_REQUIRED** , **REJECTED**).
-

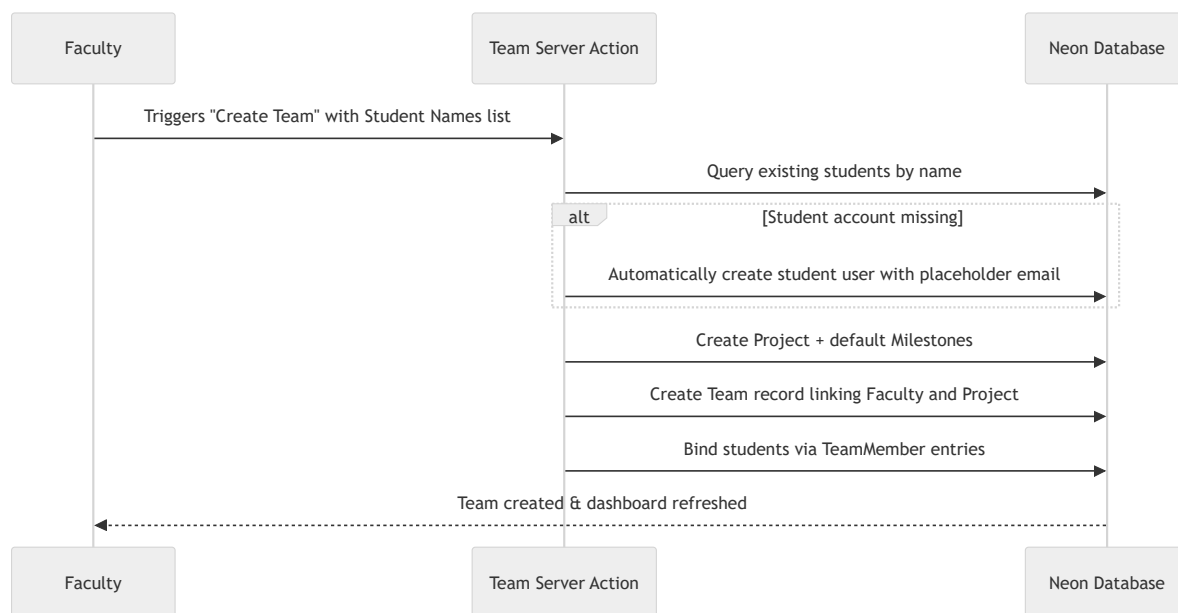
3. Core Operational Workflows

A. Authentication & Onboarding Workflow

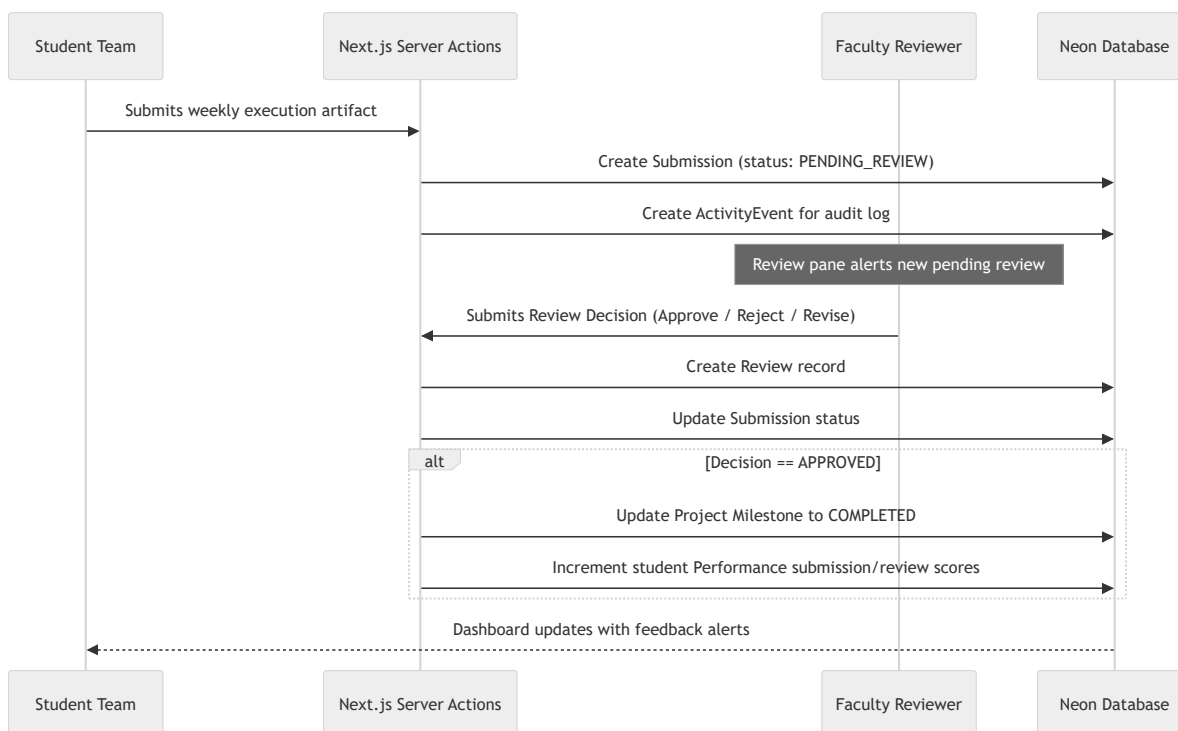


- **Schema Validation:** Server validates client inputs with `registerSchema` , allowing `null` values on optional fields (like `specialization` or `phoneNumber`) to adapt to different user roles (Student vs. Faculty).
- **Self-Service Rules:** The platform restricts direct onboarding registrations on public endpoints to Student profiles only.

B. Project & Team Creation Workflow



C. Submission & Review Cycle



4. Security & Access Control (RBAC)

Authorization is implemented as a layered middleware:

1. **Session Control:** NextAuth JWT validation checks if the request session exists.
2. **Role Rules:** Middleware blocks path prefixes (e.g. `/faculty/*` , `/admin/*`) based on the authenticated user's role:

```
export function hasAllowedRole(userRole: Role, allowedRoles: Role[]) {  
  return allowedRoles.includes(userRole);  
}
```

3. **Data Scope Isolation:** Queries verify that data is scoped. For example, a Faculty member can only fetch teams where `facultyId === session.user.id` .
-

5. Performance Optimizations

To ensure fast page load times and resolve slow dashboard transitions, several optimizations are active:

A. Single Round-Trip Joins (`relationLoadStrategy`)

By default, Prisma executes multiple SELECT queries for relations. We use `relationLoadStrategy: "join"` to let PostgreSQL compile relation loads into a single query with SQL Joins:

```
const record = await db.user.findUnique({
  where: { id: userId },
  relationLoadStrategy: "join",
  include: studentDashboardInclude,
});
```

B. Prisma Relation Joins Preview Feature

Configured in `prisma/schema.prisma` to enable database-side join optimizations:

```
generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["relationJoins"]
}
```

C. Database Indexes

Key indexes are declared in `schema.prisma` to keep search and filtering times low:

- **Composite index on team selection:** `@@unique([teamId, userId])` ON `TeamMember` .
- **Index on user roles:** `@@index([role])` ON `User` .
- **Foreign key indexes:** Added on all parent relations (`facultyId` , `teamId` , `projectId` , `submissionId`) to maximize JOIN performance.